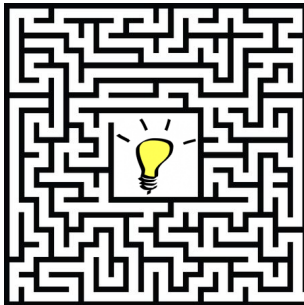


Advanced IT– Lösungsvorschläge

Prof. Dr. Henning Pagnia

DHBW Mannheim

Frühjahr 2026



Aufgabe 1 (man, cat)

- (a) Welche Aufgaben erfüllt der Befehl `cat`?
- (b) Erzeugen Sie mittels `cat` zunächst zwei kleine Dateien mit jeweils 4 Zeilen und anschließend eine dritte als Zusammenschluss der ersten beiden Dateien. Dabei sollen fortlaufende Zeilennummern vorangestellt werden!

Lösungsvorschlag

(a)

```
man cat
```

(b)

```
cat >file1
```

```
cat >file2
```

```
cat file1 file2 | cat -n >file3
```

(Drücken Sie jeweils Ctrl-D zum Eingabe-Ende)

Aufgabe 2 (echo, wc)

(a) Geben Sie unter Verwendung des Befehls `echo` den folgenden Text exakt wie angegeben auf dem Bildschirm aus:

```
Dear friend!
```

```
Welcome!
```

Tipp: 'man bash' und dort nach 'echo' suchen

(b) Überlegen Sie, welche Ausgabe die folgende Befehlszeile erzeugt:

```
echo >testf; echo >>testf; wc testf
```

Lösungsvorschlag

(a)

```
echo -e 'Dear friend!\n\n\nWelcome!'
```

(b)

Es werden zwei Leerzeilen in `testf` geschrieben und anschließend die Anzahl an Bytes, Wörter und Zeilen der Datei ausgegeben.

Aufgabe 3 (sort)

Sortieren Sie die Zeilen einer Datei, so dass gleiche Zeilen in der Ausgabedatei nur ein einziges Mal vorkommen, wobei Groß- / Kleinschreibung ignoriert werden soll.

Beispieldatei 'Namen' zum Testen:

```
Mueller fehlt  
Maier fehlt  
Schmidt fehlt  
Maier fehlt  
Mueller fehlt  
Schmidt fehlt  
maier fehlt  
Schmidt fehlt  
Mueller fehlt  
schmidt      fehlt  
mueller FehLt
```

Die korrekte Ausgabe ist:

```
Maier fehlt  
Mueller fehlt  
schmidt      fehlt  
Schmidt fehlt
```

Lösungsvorschlag

```
sort -uf Namen
```

Aber nicht so:

```
sort -u Namen | sort -f
```

o. umgek.

Aufgabe 4 (uniq, head)

Finden Sie die am häufigsten auftretende Zeile einer Datei (ohne Berücksichtigung der Groß- / Kleinschreibung)!

Tipp: Verwenden Sie die Tools `sort`, `uniq` und `head`

Lösungsvorschlag

Die gewünschten Parameter sind '-ci' .

```
sort -f Namen | uniq -ci | sort -rn | head -n1
```

Zuerst wird die Datei sortiert (wichtig!), danach werden alle aufeinanderfolgenden identischen Zeilen gezählt, wobei die Groß- / Kleinschreibung ignoriert wird. Dann wird nochmal sortiert, so dass die häufigste Zeile ganz oben steht und schließlich nur diese ausgegeben.

Aufgabe 5 (Shell-Script)

(a) Schreiben Sie ein Shell-Script `mydel`, das eine angegebene Datei löscht, jedoch zuvor eine Sicherungskopie in Ihrem Verzeichnis `$HOME/Papierkorb/` anlegt.

Achten Sie darauf, dass auch sog. *Wildcards* ('*', '?') korrekt verarbeitet und auch Verzeichnisse gelöscht werden können!

Tipp: Sie können im Shell-Script die auf der Kommandozeile übergebenen Parameter referenzieren:

\$1 bezeichnet den ersten, \$2 den zweiten, ..., \$@ alle Parameter.

(b) Erstellen Sie in Ihrem Home-Verzeichnis ein Verzeichnis `bin/` und kopieren Sie Ihr Script dorthin in die Datei `rm`. Verändern Sie Ihren Datei-Suchpfad so, dass bei der Eingabe des Kommandos `rm` Ihre Datei aufgerufen wird. Wie können Sie nun den Papierkorb leeren?

Lösungsvorschlag

(a) Datei: mydel

```
#!/bin/bash
echo Removing $@
mv $@ $HOME/Papierkorb/
```

(b) Ausführbar machen, kopieren in das Verzeichnis, anpassen des Suchpfads

```
chmod 755 mydel
cp mydel $HOME/bin/rm
export PATH=$HOME/bin:$PATH
```

(c) Leeren des Papierkorbs geht dann mit
`/bin/rm -r $HOME/Papierkorb/*`

Aufgabe 6 (read, case)

Erstellen Sie ein Script, das solange von der Tastatur einen Namen einliest, bis ein 'q' oder ein 'quit' eingegeben wird. Für jeden Namen aus der nachstehenden Liste soll die passende Note angezeigt werden. Für alle anderen Namen soll '5.0' ausgegeben werden.

carla	1.0	uta	4.0
hans	1.0	georg	4.0
berta	4.0		

Lösungsvorschlag

```
while (true);
do
    echo -n "Geben Sie bitte den Namen ein: "
    read STUDENT
    case $STUDENT in
        q | quit ) exit;;
    esac
    echo -n "Die Note von $STUDENT ist "
    case $STUDENT in
        carla | hans ) echo "1.0";;
        uta | berta | georg ) echo "4.0";;
        * ) echo "5.0";;
    esac
done
```

Aufgabe 7 (awk)

Schreiben Sie ein `awk`-Programm, das zunächst vom Benutzer zwei Wörter einliest und dann in einer Eingabedatei alle Vorkommen des ersten Wortes durch das zweite ersetzt. Geben Sie das Ergebnis auf der Standardausgabe aus!

Tipp: Mit `getline str < "-"` können Sie eine Eingabe von der Tastatur in die Variable `str` einlesen.

Lösungsvorschlag

```
BEGIN { printf("From = ") > /dev/stderr";  
        getline FROM < "-"  
        printf("To = ") > /dev/stderr";  
        getline TO < "-"  
    }  
    { for(j=1; j<=NF; j++)  
        if ($j == FROM)  
            printf("%s ", TO);  
        else  
            printf("%s ", $j);  
        printf("\n");  
    }
```

Aufgabe 8 (Threads)

Bitte ändern Sie die Klasse *MyThread1* von Folie 30 so, dass der *main*-Thread unmittelbar nach Beendigung aller Threads die Nachricht "Alle fertig." auf dem Bildschirm ausgibt. Achten Sie dabei darauf, dass auch weiterhin alle Threads nebenläufig ablaufen!

Lösungsvorschlag

```
public class MyThread extends Thread {
    private int id;    // Thread ID
    public MyThread( int id ){
        this.id = id;
    } // Konstruktor

    public void run() {
        try {
            Thread.sleep( (int) ( Math.random() * 1000 ) );
        } catch (Exception e) {}
        System.out.println( "Hello World (ID = " + id + ")" );
    } // run

    public static void main( String[] args ) {
        Thread[] t = new Thread[9];
        for (int i=0; i<9; i++)
            t[i] = new MyThread( i ).start();
        try {
            for (int i=0; i<9; i++)
                t[i].join();
            System.out.println("Alle fertig!");
        }
        catch (Exception e) {System.out.println(e);}
    } // main
} //class MyThread
```

Die Referenzen auf die Threads müssen bei deren Erzeugung in einem Array oder in einer Array-List gespeichert werden. Nachdem alle Threads gestartet wurden, wird in einer weiteren Schleife mittels `join()` auf jeden einzelnen Thread gewartet. Wenn dann alle Threads beendet sind, wird die Nachricht auf dem Schirm ausgegeben.

Aufgabe 9 (Parallele Berechnungen)

(a) Schreiben Sie ein Java-Programm, das ein Array der Größe 2 097 152 ($= 2^{21}$) mit Integer-Werten > 0 füllt. Bilden Sie dann die Gesamtsumme und geben sie diese aus. Messen Sie die Zeit in Nanosekunden, die das Programm nur für die Summenbildung benötigt hat.

(b) Wiederholen Sie das Vorgehen mit 2, 4, 8, 16, 32, 64, 128 und 256 Worker-Threads, die jeweils die Summe über einen eigenen Teil des Arrays bilden und an den Haupt-Thread zurückgeben und messen Sie ebenfalls jeweils die für die Summenbildung benötigte Zeit. Bauen Sie auch zusätzlich zur Summenbildung bei allen Threads zeitintensivere Berechnungen ein. Führen Sie verschiedene Messreihen durch!

Hinweis: Übergeben Sie eine fortlaufende ID an die Worker, so dass jeder Worker berechnen kann, welchen Ausschnitt des Arrays er bearbeiten muss.

(c) Diskutieren Sie die Ergebnisse!

Lösungsvorschlag

```
import java.util.Date;
```

```
public class SumWorker extends Thread {  
    static int MAX = 1; // Number of Worker Threads: 1,2,4,8, ...  
    static int anz = 1024 * 2048;  
    static int[] a = new int[anz];  
    static long[] erg = new long[MAX];  
    static SumWorker w[] = new SumWorker[MAX];  
    int myID = -1;  
  
    public SumWorker (int id){  
        myID = id;  
    }  
  
    public void run(){  
        double x;  
        int start = myID * (a.length / MAX);  
        int ende = (myID + 1) * (a.length / MAX);  
        long sum = 0;  
        for (int i = start; i < ende; i++){  
            sum = sum + a[i];  
            for (int j=1; j<20; j++) // sometime consuming computing  
                x = Math.log(Math.sqrt(3.1415 * (7^2) * j));  
        }  
    }  
}
```

```

    erg[myID]= sum;
}

public static void main( String[] args ) {
    // fill array
    for (int i=0; i<a.length; i++)
        a[i]= i;
    // create workers
    for (int i=0; i<MAX; i++)
        w[i] = new SumWorker(i);

    // start workers
    long date1 = System.currentTimeMillis();
    for (int i=0; i<MAX; i++)
        w[i].start();

    // wait for all workers to terminate
    try{
        for (int i=0; i<MAX; i++)
            w[i].join();
    } catch (InterruptedException e){System.err.println(e);}

    // compute result
    long sum = 0;
    for (int i=0; i<MAX; i++)
        sum += erg[i];
    long time = System.currentTimeMillis() - date1;
    System.out.println( "Used " + MAX + " Worker Threads - Result = " + sum +
        "      Computing Time = " + time );
}
}

```

Laufzeitmessungen (variieren, je nach System)

Threads	Min	Max
1	111 ms	355 ms
2	56 ms	73 ms
4	30 ms	43 ms
8	16 ms	27 ms
16	15 ms	19 ms
32	15 ms	20 ms
64	16 ms	19 ms
128	17 ms	22 ms
256	24 ms	31 ms

(Messwerte für 100 Messungen auf 2,4 GHz 8-Core Intel Core i9, 32 GB, macOS 13.5.1)

Der Aufwand für die Koordination der Threads (join ist ein System-Call) wirkt sich stärker aus je mehr Threads erzeugt werden. Bei zeitintensiveren Berechnungen, ggf. auch mit blockierenden E/A-Operationen, wirkt sich Multi-Threading i. Allg. stärker aus. Bei zu vielen Threads relativieren sich jedoch die Vorteile und die Zeiten steigen wieder an.

Aufgabe 10 (Verklemmungen)

Auf einem Betriebssystem, das Dijkstras Banker's Algorithm zur Vermeidung von Verklemmungen einsetzt, existieren die Betriebsmitteltypen **A**, **B**, **C**, **D** und die Prozesse **P₁**, **P₂**, **P₃**, **P₄**. Der aktuelle Zustand im System sei wie folgt:

Exist = (?, ?, ?, ?)

Avail = (1, 2, 3, 3)

max_needed(P_i):

	A	B	C	D
P₁ :	7	4	3	1
P₂ :	3	2	2	6
P₃ :	9	0	2	2
P₄ :	3	1	4	6

assigned(P_i):

	A	B	C	D
P₁ :	0	1	0	1
P₂ :	2	0	0	3
P₃ :	3	0	2	2
P₄ :	3	1	4	2

- Berechnen Sie den Betriebsmittelvektor **Exist** und die aktuelle max. Nachforderungsmatrix **needed(P_i)**.
- Wieso ist der resultierende Zustand unsicher, falls das Betriebssystem dem Prozess **P₃** eine weitere Instanz des Betriebsmittels **A** zuteilt?
- Ist der gegebene Zustand sicher? Falls ja, geben Sie eine der möglichen Ausführungsreihenfolgen und den resultierenden Verfügbarkeitsvektor an.

Lösungsvorschlag

(a) Betriebsmittelvektor **Exist** : (9, 4, 9, 11)

	A	B	C	D
P_1 :	7	3	3	0
$needed(P_i)$: P_2 :	1	2	2	3
P_3 :	6	0	0	0
P_4 :	0	0	0	4

(b) Unsicher, weil es keine mögliche Ausführungsreihenfolge für die weiteren Anforderungen gibt. Jeder Prozess könnte noch Betriebsmittel nachfordern wollen, bevor er sich beendet und seine Betriebsmittel freigeben würde.

(c) Ist sicher; die mögliche Ausführungsfolge ist $P_2 \Rightarrow P_4 \Rightarrow P_3 \Rightarrow P_1$.

$P_2 \Rightarrow \text{Avail} = (3, 2, 3, 6)$

$P_4 \Rightarrow \text{Avail} = (6, 3, 7, 8)$

$P_3 \Rightarrow \text{Avail} = (9, 3, 9, 10)$

$P_1 \Rightarrow \text{Avail} = (9, 4, 9, 11)$

Aufgabe 11 (Synchronisationsfehler)

Beim nebenläufigen Zugriff auf gemeinsam genutzte Datenstrukturen kann es zu Inkonsistenzen kommen. In dieser Aufgabe sollen Sie diese einmal am "lebenden System" beobachten.

- (a) Implementieren Sie in JAVA eine Klasse *FifoQueue* als eine einfach verkettete Liste, in der String-Objekte verwaltet werden können. Definieren Sie hierzu die beiden Methoden *put(String s)* sowie *String get()*.
- (b) Erzeugen Sie zwei Threads, die parallel auf der Queue arbeiten, und zeigen Sie, dass dabei durch die nebenläufigen Zugriffe von Zeit zu Zeit Fehler auftreten. (Hinweis: Verwenden Sie ggf. *Thread.yield()*-Aufrufe um eine Thread-Umschaltung zu forcieren.)
- (c) Sichern Sie die Methoden mittels eines Semaphors ab, und zeigen Sie, dass nun alles korrekt ausgeführt wird.

Lösungsvorschlag

```
import java.util.concurrent.Semaphore;
```

```
class FifoQueue {  
    public Elem head; // head element of list  
    boolean sync;     // use synchronization?  
    Semaphore sem;    // mutex Semaphore  
  
    public FifoQueue (boolean sync) {  
        head = null;  
        sem = new Semaphore(1, true);  
        this.sync = sync;  
    }  
  
    public void put (Elem newEl, int id) {  
        try {  
            if (sync) sem.acquire();  
            if ( head == null)  
                head = newEl;  
            else {  
                Elem el = head;  
                while (el.next != null) {  
                    el = el.next;  
                    // Thread.yield(); // if needed  
                }  
            }  
        }  
    }  
}
```

```

        el.next = newEl;
    }
    if (sync) sem.release();
} catch (InterruptedException e){e.printStackTrace();}
} // put

public String get (int id) {
    Elem el = null;
    try{
        if (sync) sem.acquire();
        el = head;
        if (el != null) {
            // Thread.yield(); // if needed
            head = el.next;
            el.next = null;
        }
        if (sync) sem.release();
    } catch (InterruptedException e){e.printStackTrace();}
    if (el == null) System.err.println("ERROR: get() bei leerer Queue!");
    return el.data;
} // get

public synchronized String toString() {
    String s = "";
    Elem el;
    for (el = this.head; el != null; el = el.next)
        s = s + "[" + el.data + "]" + " -> ";
    s += "NULL";
    return s;
} // toString

```

```
public void testIt (int id) {
    put(new Elem(id+":a"), id);
    put(new Elem(id+":b"), id);
    put(new Elem(id+":c"), id);
    get(id);
    get(id);
    put(new Elem(id+":d"), id);
    get(id);
    get(id);
} // testIt

public static void main (String[] args) {
    FifoQueue q;
    if (args.length == 1 && args[0].equals("sync")) {
        q = new FifoQueue(true);
        System.out.println("Using synchronization ...");
    }
    else {
        q = new FifoQueue(false);
        System.out.println("Running without synchronization ...");
    }
    FifoProcess pA = new FifoProcess(0, q);
    FifoProcess pB = new FifoProcess(1, q);
    pA.start();
    pB.start();
    try {
        Thread.sleep(700); // wait so that threads can terminate
    } catch (Exception e) {}
    System.out.println(q);
} // main
} // FifoQueue
```

```
class Elem {  
    public String data;  
    public Elem next = null;  
  
    public Elem (String data) {  
        this.data = data;  
    }  
} // Elem
```

```
class FifoProcess extends Thread {  
    int id;  
    FifoQueue q;  
    public FifoProcess (int id, FifoQueue q){  
        this.id = id;  
        this.q = q;  
    }  
    public void run () {  
        q.testIt(id);  
    } // run  
} // FifoProcess
```

Aufgabe 12 (5-Philosophen – Dijkstra 1971)

Fünf Philosophen sitzen gemeinsam an einem runden Tisch, an dem sie *unabhängig voneinander* von Zeit zu Zeit Spaghetti essen. Jeder Philosoph hat rechts neben seinem Teller nur eine Gabel, benötigt aber zum Essen zwei Gabeln, also auch die seines linken Nachbarn. Aus diesem Grund können nicht alle Philosophen gleichzeitig essen, sondern sie müssen sich bezüglich ihrer kritischen Abschnitte *Essen* synchronisieren.

- (a) Formulieren Sie die Synchronisationsbedingungen mittels eines Synchronisationsgraphen.
- (b) Geben Sie eine Java-Lösung unter Verwendung des Semaphorkonzepts an, wobei Sie die Philosophen jeweils als Threads implementieren sollten. Entwerfen Sie zunächst eine Lösung, bei der die Gabeln als Betriebsmittel aufgefasst werden.

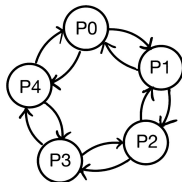
(c) Entwerfen Sie nun eine zweite Lösung unter Verwendung des Konzepts der privaten Semaphore. Gehen Sie davon aus, dass ein Philosoph zyklisch die Zustände *denkend*, *hungrig*, *essend* durchlebt.

(d) Beurteilen Sie Ihre Lösungen jeweils anhand der Kriterien Verklemmungsfreiheit, Aushungerungsfreiheit und erreichbarer Parallelitätsgrad.

Lösungsvorschlag

(a) Synchronisationsgraph

Kritische Abschnitte sind P0: „Phil. 0 isst“, P1: „Phil. 1 isst“, usw.



(b) Die Gabeln als Betriebsmittel

Gabelbelegung immer in absteigender Reihenfolge zur Vermeidung der Verklemmung mittels *Resource Ordering*:

`fork[4]` vor `fork[3]` vor `fork[2]` vor `fork[1]` vor `fork[0]`

(Zum Brechen der Symmetrie beantragen also vier Philosophen zuerst die *rechte* und dann die *linke* Gabel, der fünfte Philosoph aber umgekehrt.)

```
public class Philosoph1 {
    private Semaphore[] fork = new Semaphore[5];

    public Philosoph1 () {
        for ( int i=0 ; i<fork.length ; i++ )
            fork[i] = new Semaphore (1, true);
    }

    public void eat ( int i ) {
        // i ist der Index des Philosophen; muss zwischen 0 und 4 liegen
        if ( i==0 ) {
            fork[4].acquire();
            fork[0].acquire();
        }
        else {
            fork[i].acquire();
            fork[i-1].acquire();
        }
        // k.A.: essen;
        if ( i==0 ) {
            fork[4].release();
            fork[0].release();
        }
        else {
            fork[i].release();
            fork[i-1].release();
        }
    } // eat
} // Philosoph1
```

(c) Verwendung von privaten Semaphoren (lauffähige Lösung)

```
package AIT;
```

```
import java.util.concurrent.*;
```

```
public class PrivSemPhilos extends Thread {
```

```
    public static final int THINKING = 0;
```

```
    public static final int HUNGRY = 1;
```

```
    public static final int EATING = 2;
```

```
    int id = -1; // Philosophennummer
```

```
    // Nachfolgende Deklarationen muessen static sein!!!
```

```
    static Semaphore privsem[] = new Semaphore[5];
```

```
    static Semaphore mutex = new Semaphore(1,true);
```

```
    static private Semaphore semTrace = new Semaphore (1, true); // fuer korrekte Trace-Ausgaben
```

```
    static int state[] = new int[5];
```

```
    static PrivSemPhilos[] philo = new PrivSemPhilos[5];
```

```
    public PrivSemPhilos (int id){
```

```
        this.id = id;
```

```
        state[id] = THINKING;
```

```
        privsem[id] = new Semaphore(0,true);
```

```
    } // Konstruktor
```

```
    public void run() {
```

```
        for (int i=0; i<2; i++) {
```

```
            try {
```

```
                trace(id, "denkt");
```

```
                sleep(500+10*i); // denken simulieren
```

```
// Eintrittsprotokoll
trace(id, "will essen");
mutex.acquire();
if (state[(id+4) % 5] != EATING && state[(id+1) % 5] != EATING){
    state[id] = EATING;
    privsem[id].release();
}
else {
    state[id] = HUNGRY;
    trace(id, "muss warten");
}
mutex.release();
privsem[id].acquire(); // ggf. warten

trace(id, "isst");
sleep(500+20*id); // k.A.: essen simulieren
```

```
// Austrittsprotokoll
mutex.acquire();
trace(id, "ist satt");
state[id] = THINKING;
if (state[(id+4) % 5] == HUNGRY && state[(id+3) % 5] != EATING) {
    state[(id+4) % 5] = EATING;
    privsem[(id+4) % 5].release();
    trace(id, "weckt P-" + (id+4) % 5);
}
if (state[(id+1) % 5] == HUNGRY && state[(id+2) % 5] != EATING) {
    state[(id+1) % 5] = EATING;
    privsem[(id+1) % 5].release();
    trace(id, "weckt P-" + (id+1) % 5);
}
```

```

    }
    mutex.release();

} catch (Exception e){e.printStackTrace();}
} //for
} //run

public static void main(String[] args) throws Exception {
    for (int id=0; id<5; id++)
        philo[id] = new PrivSemPhilos(id);
    for (int id=0; id<5; id++)
        philo[id].start();           // starte Threads

    for (int id=0; id<5; id++)
        philo[id].join();
    System.out.println("*** ENDE - ALLE FERTIG ***");
} // main

private void trace(int id, String status) {
    try {
        semTrace.acquire();
        if (status.equals("denkt")){
            System.out.println ("P-"+id+" "+status);
        } else if (status.equals("will essen")){
            System.out.println ("\t\t P-"+id+" "+status);
        } else if (status.equals("muss warten")){
            System.out.println ("\t\t\t P-"+id+" "+status);
        } else if (status.equals("isst")){
            System.out.println ("\t\t\t\t P-"+id+" "+status);
        } else if (status.equals("ist satt")){
            System.out.println ("\t\t\t\t\t P-"+id+" "+status);
        }
    }
}

```

```
    } else if (status.startsWith(" weckt")){  
        System.out.println ("\\t\\t\\t\\t\\t\\t\\t      P-"+id+" "+status);  
    }  
    semTrace.release();  
} catch(Exception e) {}  
} // trace  
  
} // class
```

(c) Elegantere Implementierungsvariante

```
public class Philosoph2 {
    private static final int THINK = 1;
    private static final int HUNGRY = 2;
    private static final int EAT = 3;
    private int[] state = new int[5];
    private Semaphore[] philos = new Semaphore[5];
    private Semaphore mutex = new Semaphore(1, true);

    public Philosoph2 () {
        for ( int i=0 ; i<5 ; i++ ) {
            state[i] = THINK;
            philos[i] = new Semaphore(0, true);
        }
    }

    private void test ( int i ) {
        if ( state[(i+4) % 5] != EAT && state[(i+1) % 5] != EAT && state[i] == HUNGRY ) {
            state[i] = EAT;
            philos[i].release();
        }
    }

    public void eat ( int i ) {
        mutex.acquire();
        state[i] = HUNGRY;
        test ( i );
        mutex.release();
        philos[i].acquire();
        // k.A.: essen;
    }
}
```

```
mutex.acquire();  
    state[i] = THINK;  
    test ( (i+4) % 5 );  
    test ( (i+1) % 5 );  
    mutex.release();  
}  
  
} // Philosoph2
```

(d) Vergleich

Beide Lösungen sind verklemmungsfrei.

Lösung (b) ist aushungerungsfrei, garantiert aber nicht, dass immer auch zwei Philosophen gleichzeitig essen dürfen, wenn das möglich ist.

Lösung (c) garantiert maximale Parallelität, ist aber nicht aushungerungsfrei. (Situation: beide Nachbarn könnten immer abwechselnd aber zeitlich leicht überlappend essen.)